DESENVOLVIMENTO MOBILE · MÓDULO 02 · TRILHA FLUTTER · NÍVEL INICIANTE

Dart Essencial

Variáveis e tipos, null safety, controle de fluxo, funções, coleções, classes, mixins, records, padrões e sealed classes

Prof. Dr. Francisco dos Santos Viana

São Luís – Maranhão – Brasil · 2026

Objetivos do módulo

Ao final deste módulo você será capaz de

- Declarar variáveis com `var`, `final`, `const` e `late` e usar os tipos básicos.
- Escrever código seguro com *null safety* (`?`, `??`, `?.`, `!`).
- Usar `if`, `switch` como expressão, laços e funções com parâmetros nomeados.
- Manipular `List`, `Set` e `Map` com operações funcionais.
- Criar classes, construtores, interfaces, *mixins* e extensões.
- Modelar dados com *records*, *patterns*, `enum` e `sealed class`.

Onde praticar

`https://dartpad.dev` roda Dart (e Flutter) no navegador, sem instalar nada.

Roteiro

Neste módulo

- Fundamentos: linguagens de programação
- Primeiros passos
- Funções
- Coleções
- Orientação a objetos
- Erros
- Fechamento

Fundamentos: linguagens de programação

Paradigmas de programação

Paradigma	Ideia central
Imperativo	sequência de comandos que alteram o estado (*faça isto, depois aquilo*)
Orientado a objetos	dados e comportamentos agrupados em objetos que trocam mensagens
Funcional	funções puras, valores imutáveis, composição de funções
Declarativo	descreve **o que** se quer, não **como** obter (SQL, interfaces em Compose e Flutter)

Paradigma

Estilo de organizar o raciocínio e o código. Linguagens modernas são **multiparadigma**: Kotlin e Dart misturam imperativo, objetos e funcional.

Dica

Use o paradigma que deixa cada parte mais clara: objetos para modelar o domínio, funções para transformar dados, declarativo para descrever a interface.

Um pouco de história

Fortran (1957) era imperativo; Lisp (1958), funcional; Simula (1967) e Smalltalk (anos 1970) criaram a orientação a objetos. Kotlin (2011, JetBrains) e Dart (2011, Google) herdaram ideias de todos.

Sistemas de tipos

Tipo

Conjunto de valores possíveis e das operações permitidas sobre eles. Somar dois `Int` faz sentido; somar um `Int` a uma `Bancada`, não.

Classificação	Significado
Estática	tipos verificados na compilação (Kotlin, Dart, Java)
Dinâmica	tipos verificados na execução (Python, JavaScript)
Forte	conversões perigosas são proibidas ou explícitas
Inferência	o compilador deduz o tipo (`val x = 3`)

Por que tipagem estática ajuda

Muitos erros aparecem **antes** de o app rodar; a IDE completa código e refatora com segurança; o tipo documenta a intenção.

Solidez (*soundness*)

Um sistema de tipos é sólido quando garante que, se o programa compila, uma variável nunca terá em execução um valor de outro tipo. O Dart 3 é sólido, inclusive quanto a nulos.

O “erro de um bilhão de dólares”: o nulo

Um pouco de história

Tony Hoare introduziu a referência nula na linguagem ALGOL W em 1965. Décadas depois, chamou-a de seu “erro de um bilhão de dólares”, pelo custo das falhas que ela causou.

Null safety

O sistema de tipos separa tipos que **podem** ser nulos (`String?`) dos que **não podem** (`String`). Usar um valor possivelmente nulo sem tratar vira erro de compilação, não falha em produção.




```
if (x != null)
x ?: padrão
```

Análise de fluxo

Depois de um `if (x != null)`, o compilador sabe que `x` não é nulo naquele trecho e permite usá-lo diretamente (*smart cast* no Kotlin, promoção de tipo no Dart).

Atenção

O operador `!!` (Kotlin) ou `!` (Dart) desliga a proteção: só use quando tiver certeza.

Os pilares da orientação a objetos

Pilar	Significado e exemplo
Encapsulamento	esconder o estado interno e expor operações: a `Reserva` só muda pelo método `cancelar()`
Abstração	mostrar o essencial e ocultar detalhes: `FonteLeituras` sem dizer se é BLE ou MQTT
Herança	uma classe especializa outra: `SensorTemperatura` herda de `Sensor`
Polimorfismo	o mesmo comando, comportamentos diferentes: `fonte.leituras()` em qualquer implementação

Objeto e classe

A **classe** é o molde (atributos e métodos); o **objeto** é uma instância concreta, com estado próprio.

Composição em vez de herança

Prefira montar objetos com outros objetos ("tem um") a criar hierarquias profundas ("é um"): mudanças ficam localizadas e o código, mais flexível.

Analogia

Uma tomada é uma abstração: qualquer aparelho com o plugue certo funciona (polimorfismo), e ninguém precisa conhecer a fiação dentro da parede (encapsulamento).

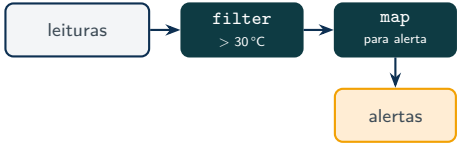
Programação funcional

Função pura

Para a mesma entrada, devolve sempre a mesma saída e não causa efeitos colaterais (não altera variáveis externas, não grava em disco, não acessa a rede).

Funções de primeira classe e de ordem superior

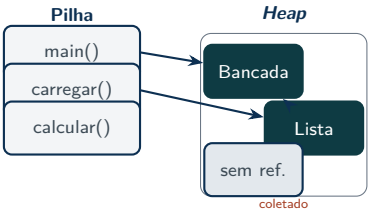
Funções são valores: podem ser guardadas em variáveis, passadas como argumento e devolvidas. Uma função que recebe ou devolve outra é de **ordem superior** (`map`, `filter`).



Por que importa no mobile

Funções puras são fáceis de testar e seguras entre *threads*; transformar listas com `map`/`filter` é mais legível que laços; interfaces declarativas tratam a tela como uma função do estado (Módulo 05).

Memória: pilha, *heap* e coleta de lixo



Pilha e *heap*

A **pilha** guarda as chamadas de função e suas variáveis locais, liberadas ao retornar. O ***heap*** guarda os objetos, que vivem enquanto houver referências a eles.

Coleta de lixo

O *runtime* encontra objetos que não são mais alcançáveis a partir das referências vivas e libera sua memória automaticamente.

Gerações

A maioria dos objetos morre jovem. Por isso o ART e a VM do Dart separam objetos novos e antigos e coletam os novos com frequência e rapidez.

Imutabilidade e igualdade

Imutável

Objeto que não muda depois de criado. Para "alterar", cria-se uma cópia com o campo novo (`copy` no Kotlin, `copyWith` no Dart).

Vantagens

- Ninguém altera o objeto "pelas costas".
- Seguro para compartilhar entre *threads* e *isolates*.
- Comparar o antes e o depois de um estado fica trivial (Módulo 07).

Igualdade de valor e de identidade

Identidade: as duas variáveis apontam para o **mesmo** objeto. **Valor**: objetos diferentes com o mesmo conteúdo. `data class` (Kotlin) e *records* (Dart) comparam por valor.

Lista mutável compartilhada

Dois componentes recebem a mesma lista; um a altera e o outro mostra dados que não esperava. Exponha listas somente leitura.

Primeiros passos

Variáveis e tipos

variaveis.dart

```dart
void main() {
  var bancada = 3;                // tipo inferido: int
  double temperatura = 24.5;
  final nome = 'Osciloscópio';    // atribui uma vez
  const limite = 30.0;            // constante de compilação
  bool ligado = true;
  late String relatorio;          // inicializada depois

  temperatura += 0.3;
  relatorio = 'Bancada $bancada: $temperatura °C';
  print(relatorio);
  print('Nome tem ${nome.length} letras');

  final texto = '''
Equipamento: $nome
Estado: ${ligado ? 'ON' : 'OFF'}''';
}
```

Palavra	Significado
`var`	mutável, tipo inferido
`final`	atribuída uma única vez, em execução
`const`	valor fixo em compilação
`late`	inicializada depois, antes do uso

Tipos básicos

`int`, `double`, `num`, `String`, `bool`, além de `List`, `Set`, `Map` e `Object`. Tudo em Dart é objeto.

Dica

Prefira `final`; use `var` só para o que muda.

Null safety

null_safety.dart

```dart
String nome = 'LDR';
// nome = null;            // ERRO de compilação

String? apelido;           // pode ser nulo
// apelido.length;         // ERRO: pode ser nulo

int? tam = apelido?.length;          // chamada segura
int tam2 = apelido?.length ?? 0;     // valor padrão
apelido ??= 'sem apelido';           // atribui se nulo

if (apelido != null) {
  print(apelido.length);   // promovido a String
}

final tam3 = apelido!.length;  // "garanto que não é
                               // nulo": evite!
```

A ideia

`String` nunca é nulo; `String?` pode ser. O compilador obriga a tratar o caso nulo antes de usar o valor.

Operador	Uso
`?.`	acessa se não for nulo
`??`	valor padrão
`??=`	atribui se for nulo
`!`	afirma não nulo (lança erro se for)

Erro comum

Espalhar `!` pelo código traz de volta os erros de nulo que o Dart tenta evitar.

Controle de fluxo

Condições

```dart
final status = temp > 80 ? 'ALERTA' : 'OK';

// switch como expressão (Dart 3)
String classificar(double t) => switch (t) {
  < 0 => 'abaixo da faixa',
  >= 0 && <= 35 => 'normal',
  _ => 'acima da faixa',
};

String descrever(Object x) => switch (x) {
  int n => 'inteiro $n',
  String s => 'texto de ${s.length} letras',
  _ => 'desconhecido',
};
```

Laços

```dart
for (var i = 0; i < 5; i++) print(i);

final sensores = ['DHT22', 'BMP280', 'LDR'];
for (final s in sensores) print(s);
sensores.forEach(print);

var tentativas = 0;
while (tentativas < 3) {
  tentativas++;
}
```

Dica

O `switch` de expressão do Dart 3 aceita *patterns* (faixas, tipos, *records*) e o compilador avisa quando faltam casos.

Funçõeses

Funções e parâmetros

funcoes.dart

```dart
double media(List<double> valores) {
  return valores.reduce((a, b) => a + b) / valores.length;
}

// corpo de expressão única (arrow)
double cParaF(double c) => c * 9 / 5 + 32;

// parâmetros nomeados (com required e padrão)
void conectar({
  required String host,
  int porta = 1883,
  bool tls = false,
}) { /* ... */ }

// parâmetros posicionais opcionais
String saudacao(String nome, [String? titulo]) =>
    'Olá, ${titulo ?? ''} $nome';

conectar(host: 'broker.local', tls: true);
```

Parâmetros nomeados

Ficam entre `{ }`, são opcionais por padrão e podem ser `required`. É assim que todos os widgets do Flutter recebem seus parâmetros: `Text('Oi', style: ...)`.

Dica

A seta `=>` substitui `{ return ...; }` quando o corpo é uma única expressão.

Funções como valores e extensões

Funções anônimas

```dart
// funções são objetos
int Function(int) dobro = (x) => x * 2;
final ehPar = (int n) => n % 2 == 0;

// recebe uma função como parâmetro
Duration medir(int n, void Function() acao) {
  final sw = Stopwatch()..start();
  for (var i = 0; i < n; i++) acao();
  return sw.elapsed;
}

final tempo = medir(1000, () => calcularFFT());
```

Extensões

```dart
extension Conversoes on double {
  String get emCelsius =>
      '${toStringAsFixed(1)} °C';
}
extension Adc on int {
  double paraVolts({double vref = 3.3,
                    int bits = 12}) =>
      this * vref / ((1 << bits) - 1);
}

print(24.567.emCelsius);   // 24.6 °C
print(2048.paraVolts());   // 1.65
```

Isto é Flutter!

`onPressed: () { ... }` é uma função anônima passada como parâmetro nomeado.

Coleções

List, Set e Map

Coleções

```dart
final nomes = <String>['DHT22', 'BMP280', 'LDR'];
final ids = {1, 2, 2, 3};               // Set: {1, 2, 3}
final pinos = {'led': 2, 'rele': 4};    // Map

nomes.add('DS18B20');
pinos['buzzer'] = 5;
final pinoLed = pinos['led'] ?? -1;     // pode ser nulo

const fixas = ['A', 'B'];   // lista imutável
// fixas.add('C');           // erro em execução

// collection if/for e spread
final menu = [
  'Painel',
  if (ehAdmin) 'Usuários',
  for (final b in bancadas) 'Bancada ${b.id}',
  ...extras,
];
```

Tipo	Característica
`List`	ordenada, aceita repetidos
`Set`	sem repetidos
`Map`	pares chave → valor

Collection if/for

`if`, `for` e `...` (*spread*) dentro de literais de lista. Muito usados para montar listas de widgets condicionalmente.

Operações funcionais

Pipeline de dados

```dart
final leituras = [
  Leitura('t1', 24.1), Leitura('t2', 81.3),
  Leitura('t1', 24.9), Leitura('t3', 19.0),
];

final alertas = leituras
    .where((l) => l.valor > 75)           // filtra
    .map((l) => '${l.sensor}: ${l.valor}') // transforma
    .toList();

final total = leituras.fold<double>(0, (s, l) => s + l.valor);
final temAlerta = leituras.any((l) => l.valor > 80);
final t3 = leituras.where((l) => l.sensor == 't3').firstOrNull;
final ordenadas = [...leituras]   // .. = cascata
  ..sort((a, b) => b.valor.compareTo(a.valor));
```

Método	Resultado
`where`	mantém os que passam
`map`	transforma cada item
`fold`, `reduce`	agrega
`any`, `every`	testes lógicos
`firstWhere`	primeiro que satisfaz
`sort`	ordena a própria lista
`toList`, `toSet`	materializa

Dica

`where` e `map` devolvem `Iterable` preguiçoso: chame `toList()` quando precisar de uma lista.

Orientação a objetos

Classes e construtores

equipamento.dart

```dart
class Equipamento {
  final String patrimonio;
  String descricao;
  int _horasDeUso = 0;          // _ = privado ao arquivo

  Equipamento(this.patrimonio, this.descricao)
      : assert(patrimonio.isNotEmpty);

  // construtor nomeado
  Equipamento.semDescricao(String p) : this(p, '---');

  bool get precisaManutencao => _horasDeUso > 500;

  void registrarUso(int horas) => _horasDeUso += horas;
}

final osc = Equipamento('IFMA-0042', 'Osciloscópio');
osc.registrarUso(12);
```

- `this.campo` no construtor atribui direto.
- Lista de inicialização após `:`.
- Construtores nomeados: `Classe.nome()`.
- `get`/`set` para propriedades calculadas.
- Prefixo `_` = privado à biblioteca (arquivo).
- Não precisa de `new`.

Construtores `const` e `factory`

`const Classe(...)` cria objetos imutáveis compartilhados (como os widgets). `factory` pode devolver um objeto já existente ou de outro subtipo.

Herança, interfaces, mixins

Hierarquia

```dart
abstract class Medidor {
  String get unidade;
  double ler();
  String formatado() => '${ler()} $unidade';
}

mixin Calibravel {
  double offset = 0;
  void calibrar(double referencia, double lido) =>
      offset = referencia - lido;
}

class Sensor { final int pino; Sensor(this.pino); }

class Termistor extends Sensor
    with Calibravel implements Medidor {
  Termistor(super.pino);
  @override String get unidade => '°C';
  @override double ler() => 25.0 + offset;
}
```

Regras

- `extends`: herança simples.
- `implements`: toda classe é também uma interface.
- `with`: *mixin* reaproveita código em várias classes.
- `abstract`: não pode ser instanciada.

Dica

`@override` documenta a sobrescrita e o analisador avisa se o método não existir na superclasse.

Enum, records e patterns

Enum aprimorado

```dart
enum StatusBancada {
  livre('verde'),
  reservada('âmbar'),
  manutencao('vermelho');

  const StatusBancada(this.cor);
  final String cor;
  bool get disponivel => this == livre;
}

print(StatusBancada.values);
```

Records e desestruturação

```dart
// record: vários valores sem criar classe
(double, double) faixa() => (10.0, 40.0);
({double min, double max}) faixa2() =>
    (min: 10, max: 40);

final (min, max) = faixa();
final (:min2, :max2) = (min2: 1, max2: 2);

// pattern em JSON
if (json case {'id': int id, 'nome': String n}) {
  print('Bancada $id: $n');
}
```

Records e *patterns* (Dart 3)

Records agrupam valores com tipo, sem criar uma classe. *Patterns* desestruturam e validam dados, muito úteis para ler JSON com segurança.

Sealed classes: modelando estados

resultado.dart

```dart
sealed class Resultado {}
class Carregando extends Resultado {}
class Sucesso extends Resultado {
  final List<Leitura> dados;
  Sucesso(this.dados);
}
class Erro extends Resultado {
  final String mensagem;
  Erro(this.mensagem);
}

// switch exaustivo: sem default
String render(Resultado r) => switch (r) {
  Carregando() => 'Carregando...',
  Sucesso(:final dados) => '${dados.length} leituras',
  Erro(:final mensagem) => 'Falha: $mensagem',
};
```

Por que `sealed`?

Todos os subtipos ficam no mesmo arquivo; o compilador conhece a lista completa e exige que o `switch` trate todos.

É exatamente o padrão usado para os estados de tela (carregando, sucesso, erro) nos Módulos 07 e 08.

Modificadores de classe

`final`, `base` e `interface` controlam quem pode estender ou implementar a classe fora da biblioteca.

Erros

Exceções

Tratando falhas

```
class SensorDesconectado implements Exception {
  final String id;
  SensorDesconectado(this.id);
  @override
  String toString() => 'Sensor $id desconectado';
}

double lerSensor(String id) {
  if (!conectado) throw SensorDesconectado(id);
  return 24.5;
}

try {
  final v = double.parse(entrada);
  print(lerSensor('t1') + v);
} on FormatException {
  print('valor inválido: $entrada');
} on SensorDesconectado catch (e) {
  print(e);
} finally {
  fecharArquivo();
}
```

`on` *versus* `catch`

`on Tipo` trata um tipo específico; `catch (e, pilha)` dá acesso ao objeto e à pilha de chamadas.

Dica

`double.tryParse(texto)` devolve `null` em vez de lançar exceção. Ótimo para validar campos de formulário.

`Exception` *versus* `Error`

`Exception`: falhas esperadas (rede, formato) que devem ser tratadas. `Error`: bugs de programação, que devem ser corrigidos.

Fechamento

Resumo

Resumo

- Prefira `final`; `const` para valores de compilação; `late` com cuidado.
- `T?` marca valores anuláveis; use `?.`, `??` e promoção de tipo; evite `!`.
- `switch` de expressão com *patterns* substitui cadeias de `if`.
- Parâmetros nomeados e funções anônimas são a base da API do Flutter.
- Coleções com `where`/`map`/`fold` e *collection if*/*for*.
- Classes, *mixins*, extensões, *records* e `sealed class` para modelar o domínio e os estados.

Glossário

Termo	Significado
Paradigma	estilo de organizar o código (imperativo, objetos, funcional, declarativo)
Tipagem estática	tipos verificados na compilação
Solidez (*soundness*)	garantia de que o tipo declarado vale também na execução
Null safety	distinção entre tipos anuláveis (`String?`) e não anuláveis
Promoção de tipo	o compilador refina o tipo após uma verificação (`if (x != null)`)
Mixin	conjunto de membros reutilizado em várias classes sem herança
Record	valor anônimo e imutável com campos, comparado por valor
Pattern	forma que confere e desestrutura valores (`switch`, `if case`)
Coleta de lixo	liberação automática de objetos sem referências

Questões de revisão

1. Que vantagens a tipagem estática e sólida do Dart traz para um app que vai crescer?
2. Por que o nulo foi chamado de "erro de um bilhão de dólares"? Como o Dart o trata?
3. Qual a diferença entre `final` e `const`?
4. Dê um exemplo de encapsulamento e um de polimorfismo no LabMonitor.
5. O que é uma função pura? Por que ela é fácil de testar?
6. Para que servem `sealed class` e `switch` exaustivo ao modelar o estado de uma tela?
7. Qual a diferença entre igualdade de valor e de identidade? Como os *records* se comportam?
8. O que acontece com um objeto quando nenhuma variável aponta mais para ele?

Exercícios

Exercício 1 — Conversões

Crie extensões `cParaF`, `fParaC` em `double` e `paraVolts` em `int` (ADC de 12 bits, $V_{ref} = 3{,}3\,\text{V}$).

Exercício 2 — Coleções

Dada uma lista de `Leitura`, calcule a média por sensor (`Map<String, double>`), as leituras fora de 10–40 °C e a média móvel de 5 amostras do sensor `'t1'`.

Exercício 3 — Estados

Modele com `sealed class` os estados de uma conexão BLE (desconectado, procurando, conectando, conectado com RSSI, falha) e escreva um `switch` exaustivo que gera a mensagem de cada estado.

LabMonitor — Etapa 2

Crie `lib/domain/models/` com `Equipamento`, `Bancada`, `Leitura` e `enum StatusBancada`, mais uma lista de dados fictícios.

Para aprofundar

- Tour da linguagem: `https://dart.dev/language`
- Null safety: `https://dart.dev/null-safety`
- Patterns e records: `https://dart.dev/language/patterns`
- Modificadores de classe: `https://dart.dev/language/class-modifiers`
- Effective Dart (boas práticas): `https://dart.dev/effective-dart`
- DartPad: `https://dartpad.dev`

Respostas das questões de revisão

1. Erros de tipo aparecem na compilação, a IDE completa e refatora com segurança e, por ser sólida, a tipagem garante os tipos também em execução.
2. Porque referências nulas causaram incontáveis falhas. O Dart separa `String` de `String?` e exige tratar o nulo (`?.`, `??`, promoção de tipo) antes do uso.
3. `final` é atribuída uma vez, em tempo de execução; `const` é um valor conhecido na compilação, compartilhado e imutável.
4. Encapsulamento: `Reserva` só muda por `cancelar()`. Polimorfismo: `FonteBle` e `FonteFake` implementam `leituras()` cada uma do seu jeito.
5. Devolve sempre o mesmo resultado para a mesma entrada, sem efeitos colaterais; o teste só confere entrada e saída.
6. A `sealed class` fecha o conjunto de estados (carregando, sucesso, erro) e o `switch` exaustivo obriga a tratar todos.
7. Identidade: o mesmo objeto (`identical`); valor: o mesmo conteúdo. *Records* com os mesmos campos são iguais por valor.
8. Ele se torna inalcançável e é liberado pelo coletor de lixo em algum momento.

Fim do Módulo 02

Próximo: Módulo 03 (comum) — Engenharia de software e Git: requisitos, métodos ágeis, versionamento e colaboração.